How Does DPO Reduce Toxicity? A Mechanistic Neuron-Level Analysis

Yushi Yang[1]* **Filip Sondej**[2] **Harry Mayne**[1] **Andrew Lee**[3]† **Adam Mahdi**[1]†
[1]University of Oxford [2]Jagiellonian University [3]Harvard University

Abstract

Safety fine-tuning algorithms reduce harmful outputs in language models, yet their mechanisms remain under-explored. Direct Preference Optimization (DPO) is a popular choice of algorithm, but prior explanations, attributing its effects solely to dampened *toxic neurons* in the MLP layers, are incomplete. In this study, we analyse four language models (Llama-3.1-8B, Gemma-2-2B, Mistral-7B, GPT-2-Medium) and show that toxic neurons only account for 2.5% to 24% of DPO's effects across models. Instead, DPO balances distributed activation shifts across a majority of MLP neurons to create a net toxicity reduction. We attribute this reduction to four neuron groups, two aligned with reducing toxicity and two promoting anti-toxicity, whose combined effects replicate DPO across models. To further validate this understanding, we develop an activation editing method mimicking DPO through distributed shifts along a toxicity representation in both probe-based and probe-free settings. This method outperforms DPO in reducing toxicity while preserving perplexity across models, without requiring any weight updates. Our work provides a mechanistic understanding of DPO and introduces an efficient, tuning-free alternative for safety fine-tuning. Our code is available on dpo-toxic-neurons.

1 Introduction

The growing capabilities of large language models (LLMs) also lead to the encoding of undesirable behaviours (Gehman et al., 2020; Gallegos et al., 2024). To mitigate harmful outputs, researchers have developed fine-tuning algorithms to prioritise human-preferred responses through reward modelling (Schulman et al., 2017; Shao et al., 2024). Among these, Direct Preference Optimization (DPO) has been a popular algorithm given its simplicity to directly optimise the policy model (Rafailov et al., 2024). While these algorithms effectively reduce harmful behaviours at the output level, there is limited mechanistic understanding of how they achieve this internally. This gap limits our ability to explain their vulnerability to jailbreaks and adversarial fine-tuning (Wei et al., 2023; Yang et al., 2023; Qi et al., 2023).

Recent studies found that fine-tuning algorithms lead to superficial changes, allowing models to retain the undesirable capabilities (Jain et al., 2024; Yang et al., 2023). In particular, Lee et al. (2024) suggested that DPO reduces toxicity by dampening the activations of a few *toxic neurons* in the MLP layers. While this offers an intuitive explanation, it assumes that toxicity is localised to a small subset of neurons. However, this is a strong claim that may oversimplify how safety fine-tuning works.

In this paper, we show that this explanation is incomplete, and offer a more comprehensive analysis of DPO's mechanism across four LLMs: Llama-3.1-8B, Gemma-2-2B, Mistral-7B and GPT-2-Medium.

Toxic neurons are not enough to explain DPO. We use activation patching to isolate the role of toxic neurons, and observe only a partial drop in toxicity across models (2.5% to 24%) compared to DPO. Where, then, does the rest of DPO's toxicity reduction come from?

Four neuron groups reduce toxicity. We show that DPO induces more nuanced, distributed activation shifts across all MLP neurons than previously suggested. We identify four mutually exclusive neuron groups that consistently contribute to the toxicity reduction across models. We find that their post-DPO activation changes depend on their orientation relative to the toxicity representation. Using activation patching, we show that their combined influence can match or even exceed the toxicity reduction achieved by DPO.

*Correspondence to: yushi.yang@oii.ox.ac.uk
†Equal contribution.

Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing, pages 29524–29543
November 4-9, 2025 ©2025 Association for Computational Linguistics

Activation editing to replicate DPO. To validate our understanding, we develop a simple activation editing method to replicate DPO. Unlike the previous post-hoc patching analyses, our method does not rely on access to post-DPO activations, nor does it require weight updates or pairwise preference data. Instead, we leverage our observations to edit activations based on the orientation of MLP weights relative to a toxicity representation. This method consistently outperforms DPO across models, showing that DPO-like effects can be achieved with minimal intervention and without fine-tuning.

2 Related Work

Here we review the DPO algorithm, the Transformer MLP layers and related work on mechanisms of safety fine-tuning algorithms.

DPO algorithm. DPO is a fine-tuning algorithm designed to align LLMs with pairwise human preference data (Rafailov et al., 2024). Given $N \in \mathbb{Z}$ preference data triplets

$$\left\{\left(x^{(i)}, y_+^{(i)}, y_-^{(i)}\right)\right\}_{i=1}^{N},$$

where x is the input prompt, and y_+, y_- are pairwise preferred and non-preferred continuations, DPO fine-tunes a policy model $\pi_\theta(y_+ \,|\, x)$ that assigns a higher likelihood to $y_+^{(i)}$ compared to $y_-^{(i)}$.

The DPO loss is defined as

$$\mathcal{L}_{\text{DPO}}(\theta) = -\log \sigma\big(\beta\left(r_\theta(x, y^+) - r_\theta(x, y^-)\right)\big),$$

where σ is the sigmoid function, β is a temperature hyperparameter and r_θ is the derived reward regularised using the reference model π_{ref}, that is

$$r_\theta(x, y) = \log \frac{\pi_\theta(y \mid x)}{\pi_{\text{ref}}(y \mid x)}.$$

MLP layers. MLPs apply two linear transformations with a non-linearity σ in between:

$$\text{MLP}^\ell(\mathbf{x}^\ell) = \sigma\left(W_K^\ell \mathbf{x}^\ell\right) W_V^\ell,$$

where $W_K^\ell, W_V^\ell \in \mathbb{R}^{d_{\text{mlp}} \times d}$, d_{mlp} and d are the dimensions of MLP hidden layers and the residual stream. MLPs can be re-expressed as:

$$\text{MLP}^\ell(\mathbf{x}^\ell) = \sum_{i=1}^{d_{\text{mlp}}} m_i^\ell \mathbf{v}_i^\ell, \quad m_i^\ell = \sigma(\mathbf{k}_i^\ell \cdot \mathbf{x}^\ell), \quad (1)$$

where $\mathbf{k}_i^\ell, \mathbf{v}_i^\ell \in \mathbb{R}^d$ are the i-th row of W_K^ℓ and W_V^ℓ, respectively. For each MLP neuron i, we refer to $\mathbf{v}_i^\ell$ as its *value vector* following Geva et al. (2022) and Lee et al. (2024). The scalar $m_i^\ell \in \mathbb{R}$ is an *activation score* controlling the scaling of the value vector $\mathbf{v}_i^\ell$. This means an MLP layer writes to the residual stream d_{mlp} times, once per neuron, via the activation-weighted value vector $m_i^\ell \mathbf{v}_i^\ell$.

Recent models (Llama, Gemma, Mistral) replace MLPs with Gated Linear Units (GLUs) (Shazeer, 2020). GLUs can similarly be expressed as a weighted sum of value vectors as in (1), where each weight is determined by some non-linear activation. See Appendix A for details.

Mechanisms of safety fine-tuning algorithms. Recent studies have shown that fine-tuning induces superficial weight changes, leaving most pre-trained capabilities intact. Jain et al. (2023) found that fine-tuning on synthetic tasks produces 'wrappers', i.e. localised weight changes in later layers optimised for each task. Qi et al. (2024) found that aligned models primarily adapt their generative distribution in the first few output tokens. Wei et al. (2024) showed that pruning just 3% of targeted parameters can undo safety alignment, highlighting the brittleness of safety mechanisms. These findings suggest that safety fine-tuning reduces harmful outputs through subtle, targeted weight changes rather than large-scale rewiring.

Lee et al. (2024) studied the mechanisms of how DPO reduces toxic outputs, attributing its effects to dampened activations of a few toxic MLP value vectors. We revisit this claim and find it to be incomplete, as shown in Section 4.

3 Experimental Setup

Here we describe the methods used in this study, including the data and models, linear probes, projections and activation patching.

3.1 Data and Models

Toxicity-eliciting prompts. We use the 'challenge' subset (N=1,199) of *RealToxicityPrompts* (Gehman et al., 2020) to elicit toxic outputs from each model. This subset is designed to trigger extremely toxic completions, making it a strong testbed for safety fine-tuning algorithms.

Models. We study four pre-trained LLMs: Llama-3.1-8B (Grattafiori et al., 2024), Gemma-2-2B (Riviere et al., 2024), Mistral-7B (Jiang et al., 2023)

and GPT-2 Medium (Radford et al., 2019). GPT-2 Medium is included to compare with claims made in Lee et al. (2024). We generate toxic outputs from each LLM using greedy decoding. Appendix B provides the MLP specification for each model.

Evaluation metrics. We report three metrics: *toxicity scores* using Detoxify (Hanu, 2020), a BERT model fine-tuned for toxicity classification that assigns a likelihood score for a text being toxic; *log perplexity*, the average negative log-likelihood of generated tokens on the Wikitext-2 dataset (Merity et al., 2016); *F1 scores*, the harmonic mean of precision and recall based on token overlap on 2,000 Wikipedia sentences (Lee et al., 2024). The latter two metrics measure general language quality, where F1 complements perplexity by capturing exact token matches.

DPO training. We implement DPO using 24,576 toxicity contrastive pairs generated from Wikitext-2 prompts (Lee et al., 2024). See Appendix C for the training hyperparameters.

3.2 Per-Neuron Toxicity Contributions

We measure per-neuron contributions to toxicity by projecting activations onto linear toxicity probes. We describe how we extract the probes, validate their effects and compute per-neuron contributions.

Linear probes. To extract toxicity representations, we train linear probes W_{Toxic} to classify toxic versus non-toxic inputs for each model. The probe is trained on the final-layer residual stream $\bar{\mathbf{x}}^{L-1}$, averaged across all token positions:

$$P(\text{toxic} \mid \bar{\mathbf{x}}^{L-1}) = \sigma(W_{\text{Toxic}}\bar{\mathbf{x}}^{L-1} + b),$$

where σ is the sigmoid function, $W_{\text{Toxic}} \in \mathbb{R}^d$ is the learned probe vector. We use the *Jigsaw Toxic Comment Classification* dataset (cjadams et al., 2017), which contains 561,808 comments labelled as toxic or non-toxic.

Across the four models, all linear probes achieve over 91% test accuracy using a 90:10 train/test split (Appendix Table 11). When projected onto each model's vocabulary space via the unembedding matrix, i.e. through LogitLens (nostalgebraist, 2020), the trained probes predominantly map to toxic tokens (Table 1).

Validating linear probes. To further validate these probes represent toxicity, we apply *activation steering* (Zou et al., 2025; Panickssery et al., 2024) by subtracting a scaled probe W_{Toxic} from the final-layer residual stream $\mathbf{x}^{L-1}$ at each token position:

$$\mathbf{x}^{L-1}_{\text{steered}} = \mathbf{x}^{L-1} - \alpha W_{\text{Toxic}},$$

where α is selected to preserve language quality (perplexity and F1) of pre-trained models (see Appendix Table 11). Increasing α further reduces toxicity scores but raises perplexity (see Appendix Table 12). Table 2 shows that steering with probe consistently reduces toxicity scores across models, validating their effects in eliciting toxic outputs. We therefore include it as a baseline for toxicity reduction.

Table 1: *The four toxicity probes predominantly project to toxic tokens in the vocabulary space.* Warning: these examples are highly offensive.

Model	Top tokens projected by probes
GPT-2-355M	f*ck, c*nt, a**hole, holes, d*ck, wh*re
Llama-3.1-8B	en, kommen, F*CK, iyah, f*ck, dirty
Gemma-2-2B	rungsseite, fu*k, Fu*king, SH*T, a**hole
Mistral-7B	sh*t, f*ck, assh, bullsh*t, f*cked, a**hole

Per-neuron toxicity change via projection. To compute per-neuron contributions, we track how the toxic representation changes at each MLP neuron during DPO via its *change in projection* onto the probe:

$$\Delta_{\text{Toxic},i} = (m_i^{\text{pre}}\mathbf{v}_i^{\text{pre}} - m_i^{\text{dpo}}\mathbf{v}_i^{\text{dpo}}) \cdot \frac{W_{\text{Toxic}}}{\|W_{\text{Toxic}}\|_2}, \quad (2)$$

where $m_i^{\text{pre}}\mathbf{v}_i^{\text{pre}}$ and $m_i^{\text{dpo}}\mathbf{v}_i^{\text{dpo}}$ are the activated components of the i-th value vector before and after DPO; the activation scores m_i^{pre} and m_i^{dpo} are averaged over 20 generated tokens for all prompts in RealToxicityPrompts. This approach, known as *direct feature attribution* (Makelov et al., 2024; Arditi et al., 2024), measures how much each neuron contributes to the toxicity representation.

3.3 Activation Patching

Throughout our work, we apply *activation patching* (Zhang and Nanda, 2024) in a counterfactual manner to isolate the effect of specific neurons on toxicity scores. Namely, for a pre-trained model and a set of MLP value vectors, we set their activations to match its post-DPO counterpart, based on the mean activation of 1,199 RealToxicityPrompts and 20 generated tokens per prompt. We then measure the resulting change in the toxicity scores.

Table 2: *Toxicity (Toxic), log perplexity (PPL), and F1 scores with activation patching and editing.* Across models, patching toxic neurons—whether those with toxic tokens or the top 256—yields only a limited drop in toxicity scores than DPO (Section 4). In contrast, patching all four of our identified neuron groups matches or outperforms DPO (Section 5.2). Our activation editing method can outperform DPO, steering with probe and patching all four groups (Section 6). Green shows the editing parameters that best compete with DPO while preserving F1 scores.

Type	Intervention	GPT-2-355M			Llama-3.1-8B			Gemma-2-2B			Mistral-7B		
		Toxic	PPL	F1	Toxic	PPL	F1	Toxic	PPL	F1	Toxic	PPL	F1
Baselines	None	0.545	3.08	0.193	0.496	1.94	0.225	0.488	4.61	0.231	0.507	1.76	0.221
	Steering with probe	0.310	3.19	0.191	0.335	2.72	0.187	0.260	5.52	0.228	0.350	2.23	0.220
	DPO	0.210	3.15	0.195	0.241	2.69	0.221	0.245	5.15	0.228	0.191	2.01	0.223
Activation patching (Sec 5.2)	Patch toxic neurons	0.479	3.09	0.193	0.491	1.94	0.225	0.487	4.61	0.231	0.505	1.76	0.232
	Patch 256 neurons	0.465	3.07	0.193	0.488	1.94	0.225	0.482	4.61	0.231	0.455	1.76	0.232
	Patch TP↓	0.407	3.07	0.191	0.488	1.94	0.223	0.470	4.87	0.235	0.502	1.80	0.229
	Patch TP↓+AN↓	0.216	3.08	0.183	0.465	1.94	0.221	0.337	4.59	0.224	0.307	1.76	0.227
	Patch TP↓+AN↓+TN↓	0.194	3.08	0.170	0.391	1.94	0.208	0.307	4.59	0.217	0.238	1.81	0.218
	Patch four groups	0.139	3.08	0.170	0.278	1.94	0.207	0.260	4.58	0.213	0.138	1.78	0.209
Activation editing (Sec 6, probe-based)	$\alpha = 0.01, \beta = 0.8$	0.123	3.08	0.179	0.045	2.19	0.186	0.199	4.54	0.188	0.038	1.77	0.179
	$\alpha = 0.01, \beta = 0.6$	0.159	3.08	0.181	0.183	2.11	0.193	0.200	4.56	0.201	0.098	1.77	0.196
	$\alpha = \mathbf{0.01}, \beta = \mathbf{0.55}$	0.203	3.08	0.183	0.241	1.96	0.196	0.216	4.56	0.210	0.125	1.77	0.202
Activation editing (Sec 6, probe-free)	$\alpha = 0.01, \beta = 0.8$	0.139	3.08	0.176	0.116	5.82	0.200	0.218	4.54	0.180	0.057	1.77	0.191
	$\alpha = \mathbf{0.01}, \beta = \mathbf{0.6}$	0.238	3.08	0.178	0.258	2.28	0.210	0.216	4.57	0.203	0.162	1.77	0.200
	$\alpha = 0.01, \beta = 0.55$	0.282	3.08	0.180	0.318	2.24	0.204	0.250	4.58	0.198	0.239	1.77	0.201

4 Toxic Neurons Are Not Enough

We start by revisiting the claims in Lee et al. (2024): (a) DPO reduces toxicity primarily by dampening the activation of toxic neurons, (b) this arises from shifts in earlier layer weights. We show here that (a) only partially explains the drop in toxicity, and in Section 5, we show that the weight shifts (b) are more nuanced than simply bypassing toxic neurons.

We measure the effect of dampening toxic neurons. We define toxic neurons by adapting the method of Lee et al. (2024): we identify the top N (= 256)[1] MLP value vectors with the highest cosine similarity to the toxic probe W_{Toxic}. In a second variant, we identify a smaller subset of interpretable value vectors. To do so, we unembed each value vector and consider it as toxic if any of its top-10 nearest tokens are toxic. We adopt LLM-as-a-judge (Zheng et al., 2023) using GPT-4o (OpenAI, 2024) to evaluate whether a token is considered toxic (e.g. curse words, slurs, sexual content). See Appendix Table 14 for the tokens projected by these toxic value vectors.

We then counterfactually isolate their effect on toxicity scores using activation patching (Section 3.3). Namely, for each pre-trained model, we set the activations of toxic value vectors to that of its post-DPO counterpart.

Table 3 reports the number of toxic neurons per model and the percentage reduction in toxicity scores through patching. Toxic neurons comprise fewer than 0.05% of all MLP neurons, and account for as little as 2.5% to 24% of the reduction in toxicity scores depending on the model. As patching captures interactions between toxic and non-toxic neurons, these results suggest that toxic neurons only account for a small portion of DPO's effect, rendering Lee et al. (2024)'s claim that DPO primarily dampens toxic neurons as incomplete.

5 A Deeper Look at DPO Weight Shifts

Here, we show that the weight shifts from DPO are more nuanced than simply bypassing toxic neurons.

5.1 DPO Balances Opposing Effects

Across all models, DPO makes minimal adjustments to the MLP weights. All MLP value vectors have a cosine similarity of 0.99 before and after DPO, likely due to the KL divergence regularisation (Rafailov et al., 2024). However, these small weight changes ($\mathbf{v}_i^{\text{pre}} \approx \mathbf{v}_i^{\text{dpo}}$) accumulate and in-

[1]This number is based on Lee et al. (2024)'s number (128). We double the number of accommodate larger model sizes, but see similar results with the original 128 vectors.

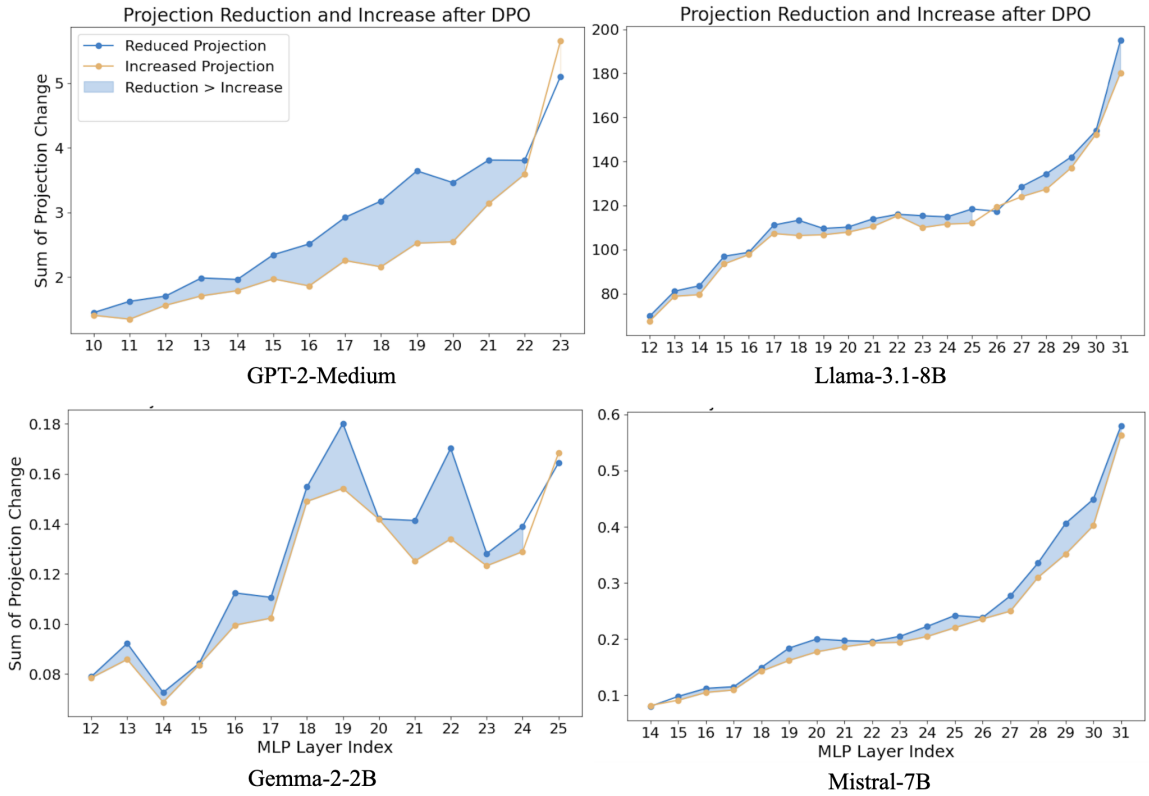

Figure 1: *DPO balances opposing toxicity writing across MLP layers.* Blue dots show total projection reduction per layer, orange dots show the total increase, both after DPO. The shaded blue areas illustrate how these opposing effects cancel out and lead to a net toxicity reduction. Projection changes grow with layers when measured against last-layer probe. Net changes in first ≈ 10 layers are negligible and omitted; see Appendix Table 5 for the full graph.

Table 3: *The number of toxic neurons per model and percentage decrease in toxicity scores after patching them.* The first row reports the number of toxic neurons with toxic tokens. The second row reports the top 256 toxic-aligned neurons. The percentage decrease is the proportion of toxicity score reduction from patching toxic neurons, relative to the total reduction of DPO (see Table 2 for the full scores).

GPT-2 355M	Llama 3.1-8B	Gemma 2-2B	Mistral 7B
59 (19.7%↓)	7 (1.96%↓)	3 (0.41%↓)	14 (0.63%↓)
256 (23.9%↓)	256 (3.14%↓)	256 (2.47%↓)	256 (16.5%↓)

duce distributed activation shifts ($m_i^{\text{pre}} - m_i^{\text{dpo}}$) across *all* MLP neurons. The majority of neurons undergo average shifts ranging from 0.66% (Llama-3.1-8B) to 16.71% (Mistral-7B), with substantial variation in the tails (see Appendix Figure 4).

These distributed activation shifts lead approximately half of all neurons (52%∼58% across models) reducing their projection onto the toxic direction ($\Delta_{\text{Toxic},i} > 0$) and the other half increasing it ($\Delta_{\text{Toxic},i} < 0$) (see Appendix Table 18). Figure 1 illustrates how the opposing neuron effects accumulate and balance out at each MLP layer, resulting in a net toxicity reduction. This suggests that DPO does not simply suppress toxic signals, but rather delicately redistributes them, balancing a trade-off across all MLP neurons.

5.2 Four Neuron Groups Reduce Toxicity

Based on these results, we study value vectors that *reduce* toxic projections ($\Delta_{\text{Toxic},i} > 0$), as they likely contribute to toxicity reduction during DPO. We categorise them into four mutually exclusive groups, and study their collective effect.

Table 4 defines the four neuron groups, categorised by their alignment with the toxicity probe (**T**oxic-aligned vs. **A**nti-toxic-aligned) and their pre-DPO activations (**P**ositive vs. **N**egative). Namely, TP ↓, TN ↓ have positive alignment with toxicity, while AP ↓, AN ↓ have negative alignment. All groups reduce toxicity projection during DPO (↓). Table 5 shows the proportions of neurons in each group across models. Note that Lee et al. (2024) only considers the neurons in TP ↓.

Figure 2c visualises how these four groups re-

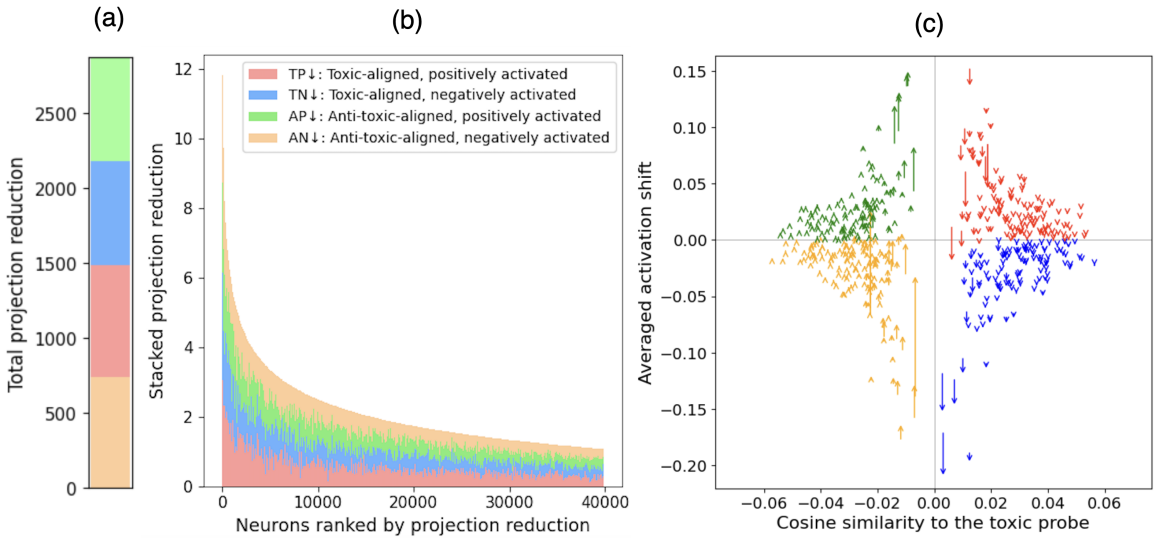

Figure 2: *Four neuron groups collectively reduce toxicity during DPO, shown for Llama-3.1-8B.* The same four groups emerge consistently across models, with panels (a) and (b) showing slightly different patterns for the other three models (see Appendix Figure 6). (a) Proportion of toxicity reduction per group, showing balanced contributions; (b) Cumulative toxicity reduction for top 40,000 neurons (ranked by reduction in projection), where groups show similar reduction rates; (c) Per-group activation shifts during DPO for the top 2,000–2,500 neurons, where each group shifts according to their orientation relative to the toxic representation.

duce toxicity writing via activation shifts for Llama-3.1-8B, with similar patterns observed in all models (see Appendix Figure 6). The activations of each group are shifted in accordance to their orientation with respect to the toxic probe. Namely, toxic-aligned weights (TP ↓, TN ↓) drop in activations, while anti-toxic aligned weights (AN ↓, AP ↓) increase in activations (promotion of "anti-toxicity").

Table 4: *Definitions of four neuron groups reducing toxicity projections* ($\Delta_{\text{Toxic, i}} > 0$). *Alignment with probe* (T vs. A) indicates whether the neuron's value vector $\mathbf{v}$ aligns positively or negatively with the toxic probe W_{Toxic} ($\mathbf{v} \cdot W_{\text{Toxic}} > 0$ or $\mathbf{v} \cdot W_{\text{Toxic}} < 0$).

Group	Alignment with probe	Pre-DPO activation	Projection change
TP ↓	Toxic-aligned	Positive	Reduced (↓)
TN ↓	Toxic-aligned	Negative	Reduced (↓)
AP ↓	Anti-toxic-aligned	Positive	Reduced (↓)
AN ↓	Anti-toxic-aligned	Negative	Reduced (↓)

Anti-toxic value vectors. What do "anti-toxic" value vectors encode? Geometrically, some anti-toxic value vectors essentially lie at the antipode of toxic semantic clusters. Namely, we take value vectors with highest cosine similarity scores to $-1 \times W_{\text{Toxic}}$ (i.e. anti-toxic). We then multiply these value vectors by -1, unembed them, and inspect their nearest tokens. Table 6 shows examples of toxic tokens they project to (see Appendix Table 15 for more examples). This shows how DPO promotes anti-toxicity by increasing the activation of anti-toxic AN ↓, AP ↓ neurons.

Table 5: *Proportions of four-neuron-group among all neurons reducing toxicity projection (↓).* Proportions are more balanced across larger LLMs. The *Sum* column shows the total number of neurons per model.

Model	TP ↓	TN ↓	AP ↓	AN ↓	Sum
GPT-2-355M	6.9%	39.1%	3.2%	50.9%	57,501
Llama-3.1-8B	25.4%	24.4%	24.6%	25.5%	239,460
Gemma-2-2B	28.8%	21.3%	21.3%	28.6%	123,898
Mistral-7B	29.7%	20.3%	20.2%	29.8%	238,236

Table 6: *Examples of anti-toxic value vectors (with reversed signs) that project to toxic tokens via Logit Lens.* Warning: these examples are highly offensive.

Model	Vector	Top tokens
GPT2	$-1 \times \mathbf{v}_{11}^{1307}$	d*mn, darn, kidding, freaking, piss
Llama3	$-1 \times \mathbf{v}_{25}^{14671}$	f*ck, f*cked, f*cking, sh*t, F*CK
Gemma2	$-1 \times \mathbf{v}_{14}^{7822}$	f*cking, godd*mn, f*ck, sh*t
Mistral	$-1 \times \mathbf{v}_{14}^{14693}$	sh*t, f*ck, Block, piss, f*cking

Why negative activations? Negatively activated neurons (including TN ↓, AN ↓) take a large portion of MLP neurons, around 50% in three larger models and 87% in GPT-2 Medium (see Appendix Table 13). This results from the modern choices of activation functions: GeLU (GPT-2), GeLU-Tanh

(Gemma), and SiLU (Llama, Mistral), which allow neurons to retain small negative activations for negative inputs (Hendrycks and Gimpel, 2023). This enables plenty of neurons to maintain gradient flow and contribute marginally to the toxicity representation through their activation shifts.

Four groups reduce toxicity at different rates. When ranking neurons by their reduction of toxicity projection, the four groups show different reduction rates. In Llama-3.1-8B, all groups contribute evenly, maintaining balanced shares of top-ranked neurons (Figure 2b). In contrast, in the other three models, TP $\downarrow$ dominating the top-ranked neurons, while AN $\downarrow$ gradually gains influence in later ranks—a trend most evident in GPT-2-Medium (see Appendix Figure 6). As a result, TP $\downarrow$ and AN $\downarrow$ dominate their overall toxicity reduction.

Reduction peaks in later layers. We also observe an overall increasing trend in toxicity reduction across MLP layers (see Appendix Figure 8). This shows that the four groups collectively steer each layer away from toxicity, with later layers giving the strongest suppression of toxic outputs. This upward trend may be partly due to the probes being extracted from the final layer.

Activation patching confirms the collective effects of four groups. Finally, we confirm the collective effect of the four groups using activation patching. This post-hoc analysis assumes access to each group's activations after DPO and evaluates their effects counterfactually by patching each neuron group, one at a time, in the pre-trained model to match their post-DPO activations.

Table 2 shows that sequentially patching each group further reduces toxicity scores across all models, confirming each neuron group's contribution to DPO's effects. Furthermore, patching all four groups either surpasses or closely matches DPO's toxicity reduction and consistently outperforms probe-based steering. It has minimal impact on perplexity and only slightly reduces F1 scores across models. This patching outperforms DPO likely because it excludes neurons that increase toxicity projection after DPO (Section 5.1). As a sanity check, patching all neurons that increase toxicity projection ($\uparrow$) during DPO leads to higher toxicity scores across models, consistent with the projection changes (see Appendix Table 19).

6 Activating Editing to Replicate DPO

Based on our insights, we demonstrate two simple methods to replicate DPO's effects by directing editing activations. These methods only rely on a toxicity representation (e.g. a probe) and do not require any weight updates nor a pairwise preference dataset, which is not always readily available. Unlike the previous activation patching analyses, here we do not assume access to post-DPO activations.

6.1 Probe-based Activation Editing

Previously, we focused on neuron groups with reduced toxicity projections (i.e., $\Delta_{\text{Toxic, i}} > 0$) (Section 5.2). However, knowing whether a neuron increases or decreases in toxicity projection requires access to post-DPO activations (see Equation 2). To remove this dependency, we re-categorise the neuron groups based solely on their alignment with the toxicity probe and their pre-DPO activations, and do not consider their projection changes (hence notated as TP as opposed to TP $\downarrow$).

Given our new neuron groups (TP, TN, AP, AN), we leverage two key insights learned from DPO: activation shifts are distributed across all neurons (Section 5.1), and the direction of activation shifts for toxicity reduction depends on the orientation of the value vector (Section 5.2, Figure 2c).

Follow these insights, we sample a fraction β (%) of neurons from each group and minimally adjust their activations. For toxicity-aligned groups (TP, TN), we slightly decrease their activations by a factor of α (%), while for anti-toxicity-aligned groups (AP, AN) we slightly increase them. As TN and AN have negative activations, we flip the sign of α accordingly:

$$m^{\text{edit}}_{\text{TP}_\beta} = (1-\alpha) m^{\text{pre}}_{\text{TP}_\beta}; \quad m^{\text{edit}}_{\text{TN}_\beta} = (1+\alpha) m^{\text{pre}}_{\text{TN}_\beta}$$
$$m^{\text{edit}}_{\text{AP}_\beta} = (1+\alpha) m^{\text{pre}}_{\text{AP}_\beta}; \quad m^{\text{edit}}_{\text{AN}_\beta} = (1-\alpha) m^{\text{pre}}_{\text{AN}_\beta}$$

where TP_β, AN_β, TN_β, and AP_β denote the β-fraction of neurons in each group, and m^{pre} are their pre-trained activations. Again, here we do not rely on any post-DPO information (i.e., m^{DPO}).

Table 2 shows the results for selected hyperparameters α and β. These hyperparameters reflect our insights: most neurons (high β value) undergo small shifts (small α value). We find that selecting the top-β fraction of neurons ranked by cosine similarity with the toxicity probe is most effective in reducing toxicity scores. In particular, selecting $\beta = 55\%$ provides the best trade-off between toxicity reduction and F1 preservation, consistent of our

earlier finding that DPO reduces toxicity writing in roughly half of all neurons (Section 5.1). This approach outperforms both DPO and probe-based steering in toxicity reduction while preserving perplexity across pre-trained models, with only a slight F1 score decrease. Further increasing β (e.g., to 0.8) leads to greater toxicity reduction at the cost of F1 drops. Alternative sampling strategies for selecting the top-β neurons (e.g., by ascending absolute activation values) yield similar toxicity reduction across models (see Appendix Table 19).

6.2 Probe-free Activation Editing

While the previous activation editing method does not require pairwise preference data, it still relies on a latent toxicity representation, for which we use our probe. While a probe does not require pairwise preference data, it still requires labelled classification data (Section 3.2).

Here, we demonstrate that activation editing can be performed even without a probe by leveraging an alternative toxicity representation. Namely, prior works have observed a close relationship between concept representations in the model's hidden layers and the token embedding space (Lee et al., 2025). Similarly, we find that toxic tokens are nearest neighbors to our probes in the token embedding space (Table 1). Motivated by this, we replace the probe with a contrastive vector derived directly from token embeddings.

To construct this vector, we simply select sets of toxic and non-toxic token embeddings for each model and compute the difference between their mean embeddings (Table 7). This bypasses the need to train a probe model. We then apply the same activation editing method as described above.

Table 7: *Toxic and non-toxic tokens for computing the contrastive vector.* The contrastive vector is obtained by subtracting the mean embedding of non-toxic tokens from that of toxic tokens.

Toxic	`fu*k`	`sh*t`	`cr*p`	`da*n`	`a**hole`
Non-toxic	`hello`	`thanks`	`friend`	`peace`	`welcome`

The last rows of Table 2 show that this probe-free approach achieves results comparable to the probe-based method. Together, these results validate our understanding of DPO and offer a proof-of-concept alternative when weight updates are prohibitively costly or training data is not readily available.

7 Discussion and Conclusion

Our work provides a mechanistic understanding of how DPO reduces toxicity across four LLMs. Using activation patching, we showed that prior explanations are incomplete (Lee et al., 2024): a small set of toxic neurons associated with toxic tokens cannot fully account for DPO's effects. This explanation also relies on a monosemantic view of neurons, an assumption disputed by prior work (Elhage et al., 2022). Instead, DPO induces distributed activation shifts across all MLP neurons, leading to a net reduction in toxicity.

To characterise these distributed effects, we identified four neuron groups that play distinct roles in toxicity reduction and show that their combined effect replicates that of DPO. Building on these insights, we developed an activation editing method that mimics DPO by applying distributed activation shifts along a learned toxicity representation. We explored two options for this representation: a probe model and a contrastive vector derived from token embeddings. This method outperforms DPO in reducing toxicity while preserving perplexity, all without any weight updates.

DPO's tendency to spread activation shifts thinly across the network suggests that pre-trained harmful capabilities are merely thinly masked. As a result, small disruptions anywhere in the model, not just in toxic neurons, can potentially breach the safety barrier and reactivate harm. This extends prior findings on the shallowness of safety fine-tuning from the activation perspective (Jain et al., 2024; Qi et al., 2024). These distributed shifts likely arise as a by-product of regularisation to preserve pre-training performance, hinting at a deeper trade-off: the shallow safety may be an inherent cost of maintaining language quality. This diluted effect is further compounded by smooth activation functions (Section 5.2), which allow many weakly active neurons to marginally contribute to toxicity writing. This leaves much of the model's toxic capacity untapped. In fact, many MLP neurons increase their toxicity projection during DPO (Section 5.1). In contrast, our activation editing method offers a more targeted alternative by explicitly steering activations to reduce toxicity. This may explain why it achieves greater toxicity reduction than DPO, despite applying smaller average activation changes. Taken together, our findings point to the value of exploring more interpretable safety interventions as a path beyond shallow tuning.

In summary, our work provides a more complete understanding of how DPO reduces toxicity and introduces an efficient, training-free alternative.

8 Limitations

Projection to a toxic subspace. In this work, we use a linear probe to capture an aggregated toxicity representation, following common practice in the literature (Ferrando et al., 2024; Ravfogel et al., 2022). However, it may be possible that toxicity manifest along multiple directions, each capturing different aspects such as hate speech or abusive language, and thus better represented as a subspace (Uppaal et al., 2024). We conduct an initial analysis on GPT-2-Medium and find that using a subspace complicates our identification of neuron groups. We construct a toxic subspace via Singular Value Decomposition (SVD) on the top 128 toxic-aligned value vectors, where each of the top three singular vectors projects to different toxic tokens (see Appendix G). We find that most value vectors show inconsistent alignment across the three directions and mixed projection changes to the toxicity probe after DPO. A single value vector can be "toxic-aligned" in one SVD direction and "anti-toxic-aligned" in another, reducing toxicity along one axis while increasing it in another. These inconsistencies make it difficult to assign neurons to coherent groups as in our approach. We therefore leave a more robust analysis of toxic subspace projections to future work.

Assumptions for projection. We use projections to estimate each neuron's contribution to toxicity (Equation 2), assuming that neurons contribute proportionally along their activated directions. However, toxicity representations may be distributed across more complex linear combinations of neurons. Alternative tools, such as sparse autoencoders (SAEs) (Bricken et al., 2023; Cunningham et al., 2023), which learn linear feature compositions through autoencoder reconstruction, may offer a complementary perspective for tracing toxicity feature changes back to specific neurons.

Generalise the four neuron groups across tasks and models. DPO is inherently a binary algorithm, designed to train on pairwise preference data. The four neuron groups we identify naturally reflect this binary structure, where we find that their activations shift along the representation of a binary concept. We therefore expect similar neuron group structures to emerge in other binary safety-related tasks trained with DPO beyond toxicity (e.g., biased vs. unbiased content, factual vs. misinformation), a direction we leave for future work.

These neuron groups may also persist in general instruction-tuned models (e.g., those trained with supervised fine-tuning or RLHF) on binary tasks, likely also operating through distributed activation shifts due to regularisation. We leave this as another direction for exploration.

Generalise the activation editing method to more tasks. Our activation editing method requires only a linear concept representation, which can be derived from a probe or token embeddings—both relatively cheap to obtain. Future work could extend our method to other safety-related tasks (e.g., bias or misinformation) where such representations can be derived from classification data, or to general tasks where the target behaviour can be captured by representative tokens (e.g., sentiment polarity, political stance).

Acknowledgements

This work is supported by the Dieter Schwarz Foundation Project Award on AI, Government, and Policy. Computations are performed on the Brains platform at the Oxford Internet Institute.

References

Andy Arditi, Oscar Obeso, Aaquib Syed, Daniel Paleka, Nina Panickssery, Wes Gurnee, and Neel Nanda. 2024. Refusal in language models is mediated by a single direction. *Preprint*, arXiv:2406.11717.

Trenton Bricken, Adly Templeton, Joshua Batson, Brian Chen, Adam Jermyn, Tom Conerly, Nick Turner, Cem Anil, Carson Denison, Amanda Askell, et al. 2023. Towards monosemanticity: Decomposing language models with dictionary learning. *Transformer Circuits Thread*, 2.

cjadams, Jeffrey Sorensen, Julia Elliott, Lucas Dixon, Mark McDonald, nithum, and Will Cukierski. 2017. Toxic comment classification challenge. Accessed: 18-May-2025.

Hoagy Cunningham, Aidan Ewart, Logan Riggs, Robert Huben, and Lee Sharkey. 2023. Sparse autoencoders find highly interpretable features in language models. *Preprint*, arXiv:2309.08600.

Nelson Elhage, Tristan Hume, Catherine Olsson, Nicholas Schiefer, and Tom Henighan et al. 2022. Toy models of superposition. *Preprint*, arXiv:2209.10652.

Javier Ferrando, Gabriele Sarti, Arianna Bisazza, and Marta R. Costa-jussà. 2024. A primer on the inner workings of transformer-based language models. *Preprint*, arXiv:2405.00208.

Isabel O. Gallegos, Ryan A. Rossi, Joe Barrow, Md Mehrab Tanjim, Sungchul Kim, Franck Dernoncourt, Tong Yu, Ruiyi Zhang, and Nesreen K. Ahmed. 2024. Bias and fairness in large language models: A survey. *Preprint*, arXiv:2309.00770.

Samuel Gehman, Suchin Gururangan, Maarten Sap, Yejin Choi, and Noah A. Smith. 2020. Realtoxicityprompts: Evaluating neural toxic degeneration in language models. *Preprint*, arXiv:2009.11462.

Mor Geva, Avi Caciularu, Kevin Ro Wang, and Yoav Goldberg. 2022. Transformer feed-forward layers build predictions by promoting concepts in the vocabulary space. *Preprint*, arXiv:2203.14680.

Mor Geva, Roei Schuster, Jonathan Berant, and Omer Levy. 2021. Transformer feed-forward layers are key-value memories. *Preprint*, arXiv:2012.14913.

Aaron Grattafiori, Abhimanyu Dubey, Abhinav Jauhri, Abhinav Pandey, Abhishek Kadian, Ahmad Al-Dahle, Aiesha Letman, and et al. 2024. The llama 3 herd of models. *Preprint*, arXiv:2407.21783.

Laura Hanu. 2020. Detoxify. `https://github.com/unitaryai/detoxify`. Accessed: 18-May-2025.

Dan Hendrycks and Kevin Gimpel. 2023. Gaussian error linear units (gelus). *Preprint*, arXiv:1606.08415.

Samyak Jain, Robert Kirk, Ekdeep Singh Lubana, Robert P. Dick, Hidenori Tanaka, Edward Grefenstette, Tim Rocktäschel, and David Scott Krueger. 2023. Mechanistically analyzing the effects of fine-tuning on procedurally defined tasks. *Preprint*, arXiv:2311.12786.

Samyak Jain, Ekdeep Singh Lubana, Kemal Oksuz, Tom Joy, Philip H. S. Torr, Amartya Sanyal, and Puneet K. Dokania. 2024. What makes and breaks safety fine-tuning? a mechanistic study. *Preprint*, arXiv:2407.10264.

Albert Q. Jiang, Alexandre Sablayrolles, Arthur Mensch, Chris Bamford, Devendra Singh Chaplot, Diego de las Casas, Florian Bressand, and et al. 2023. Mistral 7b. *Preprint*, arXiv:2310.06825.

Andrew Lee, Xiaoyan Bai, Itamar Pres, Martin Wattenberg, Jonathan K. Kummerfeld, and Rada Mihalcea. 2024. A mechanistic understanding of alignment algorithms: A case study on dpo and toxicity. *Preprint*, arXiv:2401.01967.

Andrew Lee, Melanie Weber, Fernanda Viégas, and Martin Wattenberg. 2025. Shared global and local geometry of language model embeddings. *Preprint*, arXiv:2503.21073.

Aleksandar Makelov, George Lange, and Neel Nanda. 2024. Towards principled evaluations of sparse autoencoders for interpretability and control. *Preprint*, arXiv:2405.08366.

Stephen Merity, Caiming Xiong, James Bradbury, and Richard Socher. 2016. Pointer sentinel mixture models. *Preprint*, arXiv:1609.07843.

nostalgebraist. 2020. Interpreting GPT: The logit lens. *AI Alignment Forum*. Accessed: 18-May-2025.

OpenAI. 2024. Gpt-4o system card. *Preprint*, arXiv:2410.21276.

Nina Panickssery, Nick Gabrieli, Julian Schulz, Meg Tong, Evan Hubinger, and Alexander Matt Turner. 2024. Steering llama 2 via contrastive activation addition. *Preprint*, arXiv:2312.06681.

Xiangyu Qi, Ashwinee Panda, Kaifeng Lyu, Xiao Ma, Subhrajit Roy, Ahmad Beirami, Prateek Mittal, and Peter Henderson. 2024. Safety alignment should be made more than just a few tokens deep. *Preprint*, arXiv:2406.05946.

Xiangyu Qi, Yi Zeng, Tinghao Xie, Pin-Yu Chen, Ruoxi Jia, Prateek Mittal, and Peter Henderson. 2023. Fine-tuning aligned language models compromises safety, even when users do not intend to! *Preprint*, arXiv:2310.03693.

Alec Radford, Jeff Wu, Rewon Child, David Luan, Dario Amodei, and Ilya Sutskever. 2019. Language models are unsupervised multitask learners. *OpenAI*.

Rafael Rafailov, Archit Sharma, Eric Mitchell, Stefano Ermon, Christopher D. Manning, and Chelsea Finn. 2024. Direct preference optimization: Your language model is secretly a reward model. *Preprint*, arXiv:2305.18290.

Shauli Ravfogel, Michael Twiton, Yoav Goldberg, and Ryan D Cotterell. 2022. Linear adversarial concept erasure. In *Proceedings of the 39th International Conference on Machine Learning*, volume 162 of *Proceedings of Machine Learning Research*, pages 18400–18421. PMLR.

Morgane Riviere, Shreya Pathak, Pier Giuseppe Sessa, and Cassidy Hardin et al. 2024. Gemma 2: Improving open language models at a practical size. *Preprint*, arXiv:2408.00118.

Patrick Schober, Christa Boer, and Lothar A. Schwarte. 2018. Correlation coefficients: Appropriate use and interpretation. *Anesthesia & Analgesia*, 126(5):1763–1768.

John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. 2017. Proximal policy optimization algorithms. *Preprint*, arXiv:1707.06347.